Празно---

Съдържание

Увод 4

ГЛАВА ПЪРВА. ТЕОРЕТИЧНА ЧАСТ 5

1. Функционалност 5

1.1 Блок схема 5

2. Блок схема описане 5

2.1 Arduino Uno R4 WiFi 6

2.1.1 Характеристика на Renesas RA4M1 7

2.1.2 Характеристика на ESP32-S3-MINI-1-N8 8

2.2 Сензор BME688 9

2.3 Сензор MPU-6050 10

2.4 Захранване 11

2.5 I2C Протокол 11

3. Измервания 12

3.1 Принцип на измерванията 12

3.1.1 Измерванията на BME688 12

3.1.2 Измерванията на MPU-6050 13

4. Програмен код на системата 13

4.1 Описание на програмата 25

4.2 Функционалности на програмата 26

ГЛАВА ВТОРА. ПРАКТИЧЕСКА ЧАСТ 29

1. Облачен интерфейс – екран на наблюдател 29

1.2 Облачен интерфейс – екран с данни във времето 31

2. Инструкции за работа 33

Заключение 34

Източници 35

Увод

В условията на съвременната индустриална автоматизация и цифровизация на системите за дистанционен мониторинг на оборудването стават все по-важни за оптимизирането на производствените процеси. Чрез използването на Интернет на нещата (IoT) различни сензори и устройства могат да бъдат свързани в мрежа, което позволява в реално време да се следи състоянието на машини и процеси в предприятията. Това подпомага навременното откриване на неизправности, повишава ефективността и безопасността, както и намалява разходите за поддръжка. В рамките на този проект ще бъде разработена IoT система за дистанционен мониторинг на индустриално оборудване. Системата включва измервателни сензори за няколко типа параметри.

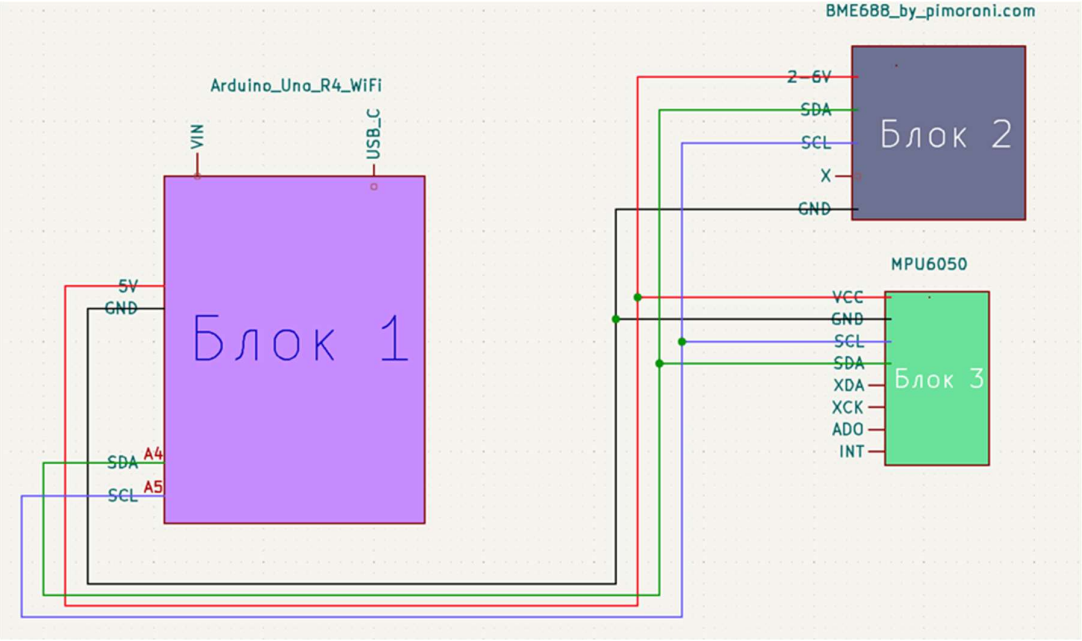
Сензорите са свързани към микроконтролер чрез I2C шина, което осигурява надеждна комуникация и лесна интеграция. След събирането на данните, микроконтролерът ги предава чрез Wi-Fi модул към облачна платформа. Там информацията се съхранява и обработва, а наблюдателят може да я наблюдава както в реално време, така и в исторически план чрез уеб интерфейс. По този начин системата осигурява централизирано проследяване на параметрите на оборудването от разстояние, без нужда от операторът да е постоянно на място. Такива системи са ключови за модернизацията на индустриалното производство. Те позволяват прогнозна поддръжка и ранно откриване на отклонения в работата на машините. С постоянната информация за температурата, вибрациите и други важни параметри операторите могат своевременно да реагират при съмнения за повреди. Това намалява риска от аварии и непланирани спирания на линиите, което води до по-висока надеждност и ефективност на производството. В настоящата дипломна работа ще бъдат разгледани основните компоненти и етапи от реализирането на системата. Ще се обсъдят хардуерните решения – избор на сензори, свързване по I2C шина и конфигурация на микроконтролера – както и комуникационните технологии (Wi-Fi и облачна платформа) за предаване на данни. Освен това ще бъде описан и разработеният уеб интерфейс за визуализация и анализ на събраната информация. В заключение ще се обобщят постигнатите резултати и ползите от внедряването на такава система в индустрията.

ГЛАВА ПЪРВА. ТЕОРЕТИЧНА ЧАСТ

1. Функционалност

Моята Internet of Things (IoT) система има за цел да осигури непрекъснато събиране на данни и тяхното изобразяване чрез уеб интерфейс, достъпен за всеки наблюдател, свързан към интернет пространството. Данните представляват обработените от микроконтролерната система измервания на физичните величини като температура, влажност, газови частици, вибрации и ъгли. Сензорните модули през I2C шината предават измерванията си към микроконтролера, който ги обработва и изпраща през Wi-Fi до облачната платформа Arduino IoT Cloud. През тази платформа се достъпва таблото за наблюдение, което показва състоянието на работната среда както в реално време, така и назад във времето.

1.1 Блок схема

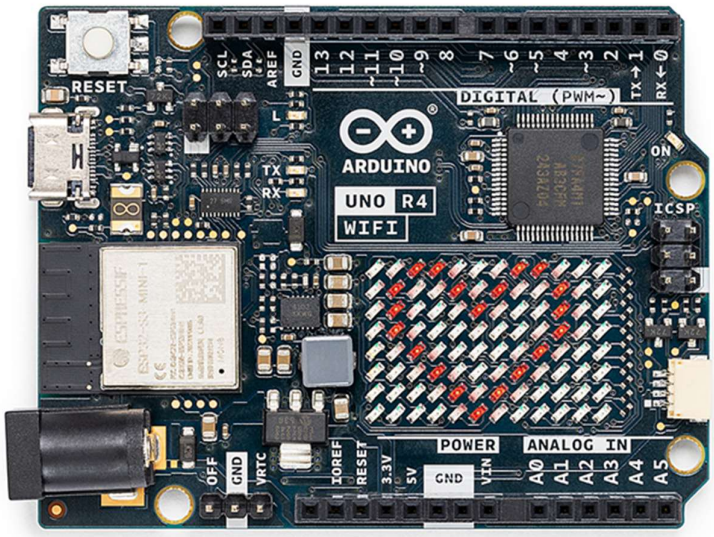


Фиг. 1 Блок схема на връзките в системата

2. Блок схема описане

На Фиг.1 са представени хардуерните компоненти Arduino Uno R4 WiFi (Блок 1), BME688 сензорен модул (Блок 2), MPU-6050 сензорен модул (Блок 3). Сензорните блокове 2 и 3 за захранени с +5V, GND изводите са свързани със общата земя на Блок 1, а SDA и SCL пиновете са свързани към A4(SDA) и A5(SCL).

2.1 Arduino Uno R4 WiFi



Фиг. 2 Снимка на Arduino Uno R4 Wi-Fi[1]

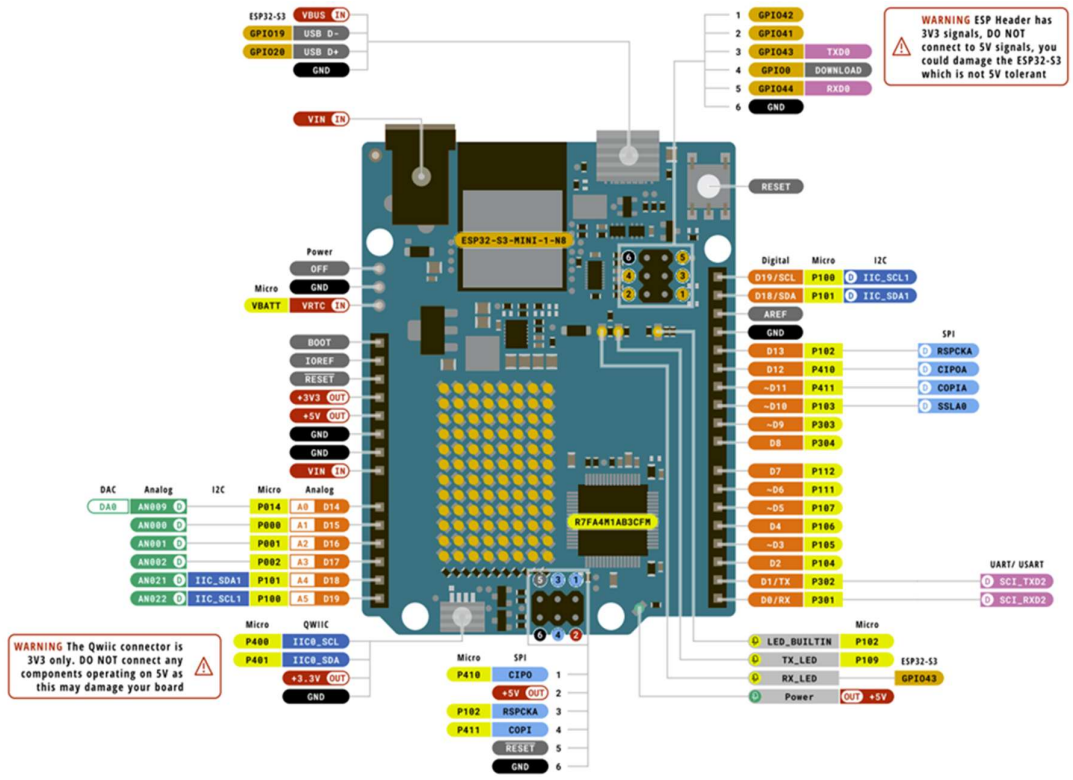
Arduino Uno R4 Wi-Fi е развойна плата от серията Arduino Uno, предназначена за разработка и прототипиране на електронни и IoT проекти.

Основната цел е да позволи улеснено използване на микроконтролерната система. Основният контролер е 32-битовия Renesas RA4M1, а допълнителният модул ESP32-S3-MINI-1-N8 е предназначен за осъществяване на безжична комуникация чрез Wi-Fi и Bluetooth.

Основните компоненти изведени и поддържани от микроконтролерите, включват 14 цифрови входно-изходни пина, 6 аналогови входни пина, 1 цифров-аналогов преобразувател(DAC) и 6 пина от цифровите с функционалност за широчинно-импулсна модулация(PWM). Поддържани комуникационни интерфейси са UART, I2C, SPI и CAN шини.

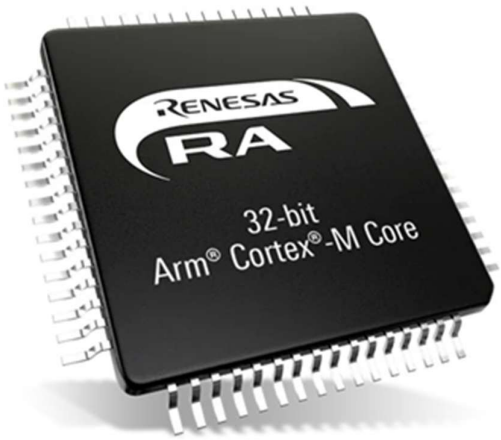
Захранването се осигурява чрез VIN порт или USB-C който също служи за програмиране и комуникация с компютър. Има вградена LED матрица като опция за визуализация.[2]

[1] [8] Espressif. [Онлайн]. Available: https://www.espressif.com/en/news/UNO_R4_WiFi_ESP32-S3.
[2] [1] Arduino, „Arduino UNO R4 WiFi Datasheet (ABX00087),“ TDK, 2023.



Фиг. 3 PINOUT диаграма за Arduino Uno R4 Wi-Fi [3]

2.1.1 Характеристика на Renesas RA4M1



Фиг. 4 Модел на как изглежда RAM1 чип [4]

[3] [1] Arduino, „Arduino UNO R4 WiFi Datasheet (ABX00087),“ TDK, 2023.
[4] [7] M. Electronics. [Онлайн]. Available: https://ar.mouser.com/new/renesas/renesas-ra8m1-32-bit-mcus/.

Renesas RA4M1 е 32-битов микроконтролер с ARM Cortex-M4F ядро, работещ с тактова честота до 48MHz и имплементиран FPU за единичноточкова плаваща аритметика. Вграденият 256kB Flash служи за съхранение на програмния код, а 32kB SRAM – за данни и временни буфери и 8kB EEPROM-стил DataFlash за продължително съхранение на данни.

RA4M1 предоставя 14-битов АЦП (до 12 канала) и 12-битов ЦАП за прецизни аналогови измервания и изходи. Управлява 14-те цифрови входно-изходни пина(6 от тях са с широчинно-импулсна модулация като възможен режим), 6-те аналогови входни пина които са АЦП с 14-битовова резолюция и единият 12-битовов ЦАП.

Поддържа комуникационните интерфейси, USB 2.0 Full-Speed, CAN 2.0B, UART/SPI/I²C multimaster шини. Друго което има са сегментен LCD контролер за HMI и капацитивен сензорен модул (CTSU) за развойни интерфейси.

ARM (“Aadvanced RISC Machines”) е вид процесорна архитектура от компанията Arm Holdings plc. ARM процесорите са с RISC (Reduced Instruction Set Computing), което означава, че използва по-малък брой инструкции. Това го прави по-бързодействащ и енерго ефективен, качества подходящи за мобилни, IoT и вградени системи. Те могат да бъдат 32-битови (например ARMv7) или 64-битови (например ARMv8 и ARMv9). [5]

2.1.2 Характеристика на ESP32-S3-MINI-1-N8



Фиг. 5 Снимка на как изглежда чипа на ESP32-S3-MINI-1-N8

[5] [2] R. Electronics, „RA4M1 Datasheet Rev.1.10,“ 2023.

ESP32-S3-MINI-1-N8 (На Фиг. 5) е модул, базиран на SoC Espressif ESP32-S3FN8, има двуядрен Xtensa 32-битов LX7 процесор с тактова честота до 240 MHz, достатъчно изчислителна мощност за IoT и Machine Learning приложения. Модулът разполага с 384 kB ROM за програмен код и 512 kB SRAM за временни данни. Процесорът поддържа SIMD (Single Instruction, Multiple Data) архитектура, която е за паралелна обработка на множество данни с една инструкция. [6]

2.2 Сензор BME688



Фиг. 6 Снимка на BME688 модула

BME688 е многофункционален сензор от Bosch Sensortec, върху развойна платка на Pimoroni. Включва всички необходими за работа елементи, логическа транслация и Qwiic/STEMMA QT конектор осигуряващ и захранването, така и I2C интерфейса за лесна интеграция.

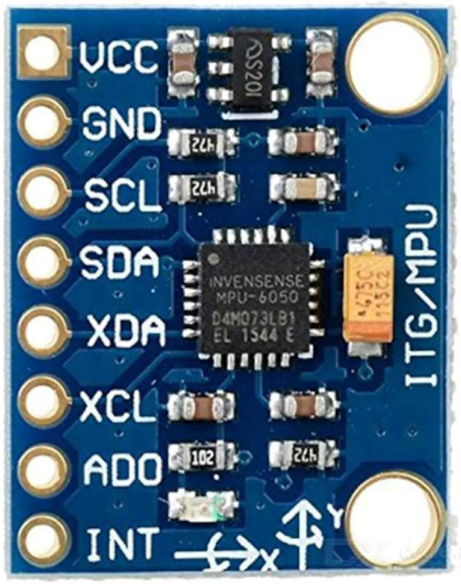
Сензорът измерва четири параметъра с висока точност. Температура в диапазона от –40 до +85 °C (±1 °C), влажност 0 до 100 % (±3 %), атмосферно налягане от 300 до 1100 hPa (±1 hPa) и концентрация на летливи органични съединения (VOCs), серни съединения (VSCs), въглероден оксид и водород в ppb.

Данните от всеки канал се четат по I2C или SPI, в ролята на Slave. На платката има припойни мостчета за смяна на адреса (0x76 или 0x77), позволяващи свързване на до два модула на една шина.

[6] [3] E. Systems, „ESP32-S3-MINI Datasheet v1.35,“ 2020.

Главната иновация на BME688 е интегрираният AI-базиран газов скенер, който може да се обучи чрез Bosch AI Studio за разпознаване на специфични газови профили.[7]

2.3 Сензор MPU-6050



Фиг. 7 Снимка на MPU-6050 модула

MPU-6050 от TDK InvenSense е многофункционален 6-осен инерционен измервателен сензор, който съчетава триизмерен акселерометър и триизмерен жироскоп в под чип. Едновременно може да измерва ускорение и ориентацията си. Разполага с вграден Digital Motion Processor (DMP), който първоначално обработва и филтрира измерванията си, намалявайки натоварването си.

Модулът може да се конфигурира на четири различни обхвата за акселерометъра: ±2 g, ±4 g, ±8 g и ±16 g с 16-битова разделителна способност. Обхватите за жироскопа са: ±250 °/s, ±500 °/s, ±1000 °/s и ±2000 °/s с 16-битова разделителна способност. Има low-pass филтри който намаляват излишният шум в сигнала, а FIFO буфер натрупа до 1 024 проби за да има време да обработи потока ако изустава.

[7] [4] B. Sensortec, „BME688 Sensor Datasheet Rev.1.3,“ 2024.

Устройството работи с напрежение VDD между 2.5V и 3.46V, както и VLOGIC (нивата на които се отчита логическа 0 или 1) напрежение от 1.8V до 3.46V, което го прави съвместимо с различни микроконтролери. Данните се предават чрез I2C

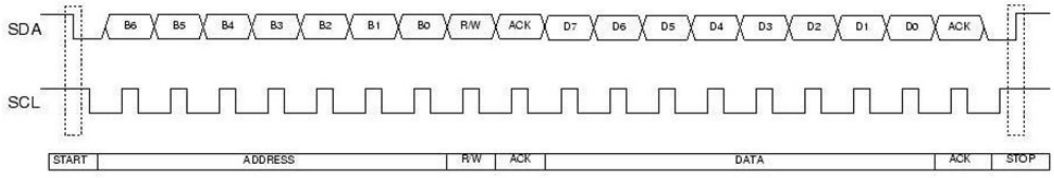
интерфейс, като модулът функционира като Slave устройство. Адресът на шината може да бъде конфигуриран (0x68 или 0x69) чрез припойни мостчета, което позволява свързване на до два модула на една и съща I2C шина. [8]

2.4 Захранване

Системата се захранва от външен 12 V адаптер, свързан към VIN на Arduino. Вграденият регулатор понижава напрежението до 5 V и 3.3 V за нуждите на контролера и сензорите.

2.5 I2C Протокол

I2C (Inter-Integrated Circuit) е двупосочен протокол за обмен данни. Всяко устройство е или Master, който инициира всяка операция, задава тактовия импулс (SCL) и адресира конкретен Slave ,или Slave, който отговаря само когато е адресиран от Master, и праща или записва данни. Необходимо е само две шини, по който успоредно може да се свържат устройствата, завършващи с Pull-up резистори който държат нормалното състояние на шината във високо състояние(или логически 1). SDA е шина за данните, работещ заедно с SCL шината която доставя тактовите импулси, необходими за синхронизация на предаването и приемането на данни по SDA. Модулите в проекта имат вградени резисторите, включително контролери които записват измерванията в регистри. Предават се в рамки (пакети) когато се извика където са записани.

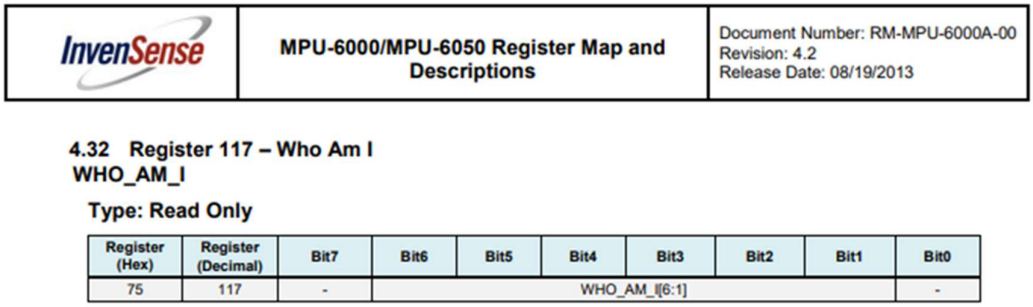


Фиг. 8 Диаграма на една рамка в I2C сигнал

[8] [6] L. M. Engineers, „MPU-6050 Arduino Tutorial,“ LME Tutorials, [Онлайн]. Available: https://lastminuteengineers.com/mpu-6050-accelerometer-and-gyroscope-arduino-tutorial/.

Всяка рамка започва с предварително дефинирано условие за старт и друго за стоп. В примера Master устройство дърпа SDA към логическа 0, докато SCL е логическа 1, и се стартира пакета. Следващите 7 бита определят адреса на който, вече спрямо осмия бит ще пише(лог. 0) или чете(лог.1). Назования Slave трябва да върне отговор с лог.0 , иначе ще се прекрати пакета. Вече след отговора се обменят данните, slave се отчита, и се подава стоп условие.

Всеки бит според производителя на сензора има инструкции какво да се напише в пакета за да направи съответното нещо. [9]В Фиг. 9 се описано прочитането на регистрите за идентификация на MPU-6050 сензора.



InvenSense | MPU-6000/MPU-6050 Register Map and Descriptions | Document Number: RM-MPU-6000A-00 Revision: 4.2 Release Date: 08/19/2013

4.32 Register 117 – Who Am I
WHO_AM_I

Type: Read Only

Register (Hex)	Register (Decimal)	Bit7	Bit6	Bit5	Bit4	Bit3	Bit2	Bit1	Bit0
75	117	-	WHO_AM_I[6:1]						-

Фиг. 9 Описание на регистър 117[10]

3. Измервания

В системата са наблюдавани общо 5 параметри температура на въздуха, влажност , качество на въздуха, вибрации, ъгъл на наклон, всеки с различен принцип по който е измерен.

3.1 Принцип на измерванията

Всеки сензор има в него включени различни преобразуватели които променят в една физична величина в електрическа. Измерва се електрическата величина и от него се правят интерпретации.

3.1.1 Измерванията на BME688

Температура на въздуха се измерва с вграден температурен сензор, което изкарва на изхода измерена температура определена температура.

[9] [9] J. Wu, „A Basic Guide to I2C,“ Texas Instruments, 2022..
[10] [5] I. (TDK), „MPU-6050 Register Map Rev.4.2,“ 2013.

Влажността се измерва с капацитивен сензор. В този случай капацитетът на сензора се променя в зависимост от влажността на въздуха. Тази промяна в капацитета се отчита и интерпретира като процент влажност.

Качеството на въздуха се определя с на нагрята металоксидант полупроводникова плочка. Промяната в съпротивлението показва наличието на определени летливи о рганични съединения (VOCs), като водород (H_2), въглероден оксид (CO) и други.

За да се получи за замърсеността на въздуха, резултатите се обработват чрез алго ритъм (на Bosch), който свързва промените в съпротивлението с ориентировъчни нива на замърсяване, базирано на предварителна калибрация или модели. [11]

3.1.2 Измерванията на MPU-6050

Вибрациите се измерват с акселерометър, който е изграден от подвижна платформа, окачена на поли-силициеви пружини. Когато върху платформата действа ускорение, тя се отклонява, което води до промяна в капацитета между платформата и фиксираните плочи. Тази промяна в капацитета се преобразува в цифров сигнал, който е пропорционален на ускорението.

Ъгълът на наклон се измерва с помощта на жироскоп, който използва принципа на Кориолисовия ефект. Вътре в жироскопа има маса, която вибрира в определена посока. Когато сензорът се завърти, Кориолисовата сила причинява отклонение на вибриращата маса. Това отклонение се измерва като промяна в капацитета, която се преобразува в цифров сигнал, пропорционален на ъгловата скорост.[12]

4. Програмен код на системата

```
// 1

#include <Adafruit_MPU6050.h>

#include <Adafruit_BME680.h>

#include "thingProperties.h"

#include <Wire.h>
```

[11] [4] B. Sensortec, „BME688 Sensor Datasheet Rev.1.3,“ 2024.
[12] [6] L. M. Engineers, „MPU-6050 Arduino Tutorial,“ LME Tutorials, [Онлайн]. Available: https://lastminuteengineers.com/mpu-6050-accelerometer-and-gyroscope-arduino-tutorial/.

```
// 2
Adafruit_MPU6050 mpu;
Adafruit_BME680 bme;

// 3
const float GAS_RESISTANCE_MIN = 1000.0;
const float GAS_RESISTANCE_MAX = 200000.0;
const float AQI_SCALE_MAX = 500.0;

// 4
float ProgramVibrationThreshold = 10.0;
float ProgramTiltThreshold = 10.0;
int ProgramMaxTemp = 100;
int ProgramMinTemp = 0;
int ProgramMaxHumidity = 70;
bool ProgramAQVeryGood = true;
bool ProgramAQNormal = false;
bool ProgramAQVeryBad = false;

// 5
void setup() {
  Serial.begin(9600);
  delay(1500);
```

```
if (!bme.begin(0x76)) {
  Serial.println("Sensor not found at 0x76");
  while (1);
}
Serial.println("BME688 Started");

if (!mpu.begin()) {
  Serial.println("Failed to find MPU6050 chip");
  while (1);
}
Serial.println("MPU6050 Started");

mpu.setAccelerometerRange(MPU6050_RANGE_8_G);
mpu.setGyroRange(MPU6050_RANGE_500_DEG);
mpu.setFilterBandwidth(MPU6050_BAND_5_HZ);
Serial.println("MPU configured");
delay(100);
bme.setTemperatureOversampling(BME680_OS_8X);
bme.setHumidityOversampling(BME680_OS_2X);
bme.setPressureOversampling(BME680_OS_4X);
bme.setIIRFilterSize(BME680_FILTER_SIZE_3);
bme.setGasHeater(320, 150);
Serial.println("BME configured");
```

```
  initProperties();
  ArduinoCloud.begin(ArduinoIoTPreferredConnection);
  delay(3000);

  setDebugMessageLevel(2);
  ArduinoCloud.printDebugInfo();
}
// 6
void loop() {
  ArduinoCloud.update();
  detectVibrationAndTilt();
  delay(500);
  readBmeSensor();
  delay(500);
  invertCloudWarnings();
}
// 7
void detectVibrationAndTilt() {
  sensors_event_t accel, gyro, temp;
  mpu.getEvent(&accel, &gyro, &temp);

  float totalAcceleration = sqrt(accel.acceleration.x * accel.acceleration.x +
                                 accel.acceleration.y * accel.acceleration.y +
```

```
                    accel.acceleration.z * accel.acceleration.z);


if (totalAcceleration > ProgramVibrationThreshold) {
  cloudVibrationWarning = true;
  Serial.println("Vibration Detected!");
  Serial.print("Total Acceleration: ");
  Serial.println(totalAcceleration);
} else {
  cloudVibrationWarning = false;
}


float tiltX = atan2(accel.acceleration.y,
                sqrt(accel.acceleration.x * accel.acceleration.x +
                    accel.acceleration.z * accel.acceleration.z)) *
            180 / PI;
float tiltY = atan2(-accel.acceleration.x,
                sqrt(accel.acceleration.y * accel.acceleration.y +
                    accel.acceleration.z * accel.acceleration.z)) *
            180 / PI;


cloudTiltX = tiltX;
cloudTiltY = tiltY;
cloudLiveVibrations = totalAcceleration;
```

```
  static unsigned long tiltStartTime = 0;


  if (abs(tiltX) > ProgramTiltThreshold || abs(tiltY) > ProgramTiltThreshold) {
    cloudTiltWarning = true;
    if (tiltStartTime == 0) {
      tiltStartTime = millis();
    } else if (millis() - tiltStartTime > 500) {
      Serial.println("-------------MPU6050--------------------");
      Serial.println("Tilt Detected!");
      Serial.print("Tilt X: ");
      Serial.println(tiltX);
      Serial.print("Tilt Y: ");
      Serial.println(tiltY);
      Serial.println("---------------------------------");


      tiltStartTime = 0;
    }
  } else {
    cloudTiltWarning = false;
    tiltStartTime = 0;
  }
}


// 8
```

```
void readBmeSensor() {
  if (!bme.performReading()) {
    Serial.println("Failed to perform reading");
    return;
  }

  bME_Temp = bme.temperature;
  bME_Index = calculateAQI(bme.gas_resistance, bme.humidity);
  bME_Humidity = bme.humidity;

  raiseAQIndicators();
  raiseTemperatureIndicators();
  raiseHumidityIndicators();
  SerialMonitorDebuggingBME();
}

// 9
void raiseAQIndicators() {
  cloudAQVeryGood = false;
  cloudAQNormal = false;
  cloudAQVeryBad = false;

  if (bME_Index >= 0 && bME_Index <= 100) {
    cloudAQVeryGood = true;
```

```
    Serial.println("Air Quality: Very Good");
  } else if (bME_Index > 100 && bME_Index <= 220) {
    cloudAQNormal = true;
    Serial.println("Air Quality: Normal");
  } else if (bME_Index > 220 && bME_Index <= 500) {
    cloudAQVeryBad = true;
    Serial.println("Air Quality: Very Bad");
  } else {
    Serial.println("Air Quality Index out of range");
  }
}

// 10
void raiseTemperatureIndicators() {
  cloudTempWarning = false;
  cloudTooLowTempWarning = false;

  if (bME_Temp > ProgramMaxTemp) {
    cloudTempWarning = true;
    Serial.println("WARNING: Temperature exceeded the upper limit!");
  } else if (bME_Temp < ProgramMinTemp) {
    cloudTooLowTempWarning = true;
    Serial.println("WARNING: Temperature fell below the lower limit!");
  } else {
```

```
    Serial.println("Temperature is within normal limits.");
  }
}
// 11
void raiseHumidityIndicators() {
  cloudHumidityWarning = false;

  if (bME_Humidity > ProgramMaxHumidity) {
    cloudHumidityWarning = true;
    Serial.println("WARNING: Humidity is above the configured upper limit!");
  } else {
    Serial.println("Humidity is within the configured range.");
  }
}
// 12
void SerialMonitorDebuggingBME() {
  Serial.println("--------------BME688--------------------");
  Serial.print("Temperature: ");
  Serial.print(bme.temperature);
  Serial.println(" °C");
  Serial.print("Humidity: ");
  Serial.print(bme.humidity);
  Serial.println(" %");
  Serial.print("Pressure: ");
```

```
  Serial.print(bme.pressure / 100.0);
  Serial.println(" hPa");
  Serial.print("Gas Resistance: ");
  Serial.print(bme.gas_resistance / 1000.0);
  Serial.println(" kOhm");
  Serial.println("---------------------------------");
}
// 13
void invertCloudWarnings() {
  cloudTempWarning = !cloudTempWarning;
  cloudTiltWarning = !cloudTiltWarning;
  cloudTooLowTempWarning = !cloudTooLowTempWarning;
  cloudVibrationWarning = !cloudVibrationWarning;
  cloudHumidityWarning = !cloudHumidityWarning;
}


// 14
float calculateAQI(float gasResistance, float humidity) {
  float aqi = 0.0;
  float gasScore = (log(gasResistance) - log(GAS_RESISTANCE_MIN)) /
                   (log(GAS_RESISTANCE_MAX) - log(GAS_RESISTANCE_MIN));
  gasScore = constrain(gasScore, 0, 1);
  gasScore = gasScore * AQI_SCALE_MAX;
```

```
  float humidityScore = 0;
  if (humidity > ProgramMaxHumidity) {
    humidityScore = (humidity - ProgramMaxHumidity) / ProgramMaxHumidity;
  }
  humidityScore = humidityScore * AQI_SCALE_MAX;


  aqi = (gasScore + humidityScore) / 2;


  return aqi;
}
// 15
void onCloudTempUpperLimitInputChange() {
  if (cloudTemp_Upper_Limit_input != ProgramMaxTemp) {
    ProgramMaxTemp = cloudTemp_Upper_Limit_input;
    Serial.println("TempUpperLimitChanged");
  }
}
// 16
void onCloudTempLowerLimitInputChange() {
  if (cloudTemp_Lower_Limit_input != ProgramMinTemp) {
    ProgramMinTemp = cloudTemp_Lower_Limit_input;
    Serial.println("TempLowerLimitChanged");
  }
}
```

```
// 17
void onCloudVibrationUpperLimitInputChange() {
  if (cloud_Vibration_Upper_Limit_input != ProgramVibrationThreshold) {
    ProgramVibrationThreshold = cloud_Vibration_Upper_Limit_input;
    Serial.println("VibrationUpperLimitChanged");
  }
}
// 18
void onCloudHumidityUpperLimitInputChange() {
  if (cloudHumidity_Upper_Limit_input != ProgramMaxHumidity) {
    ProgramMaxHumidity = cloudHumidity_Upper_Limit_input;
    Serial.println("HumidityUpperLimitChanged");
  }
}
// 19
void onCloudTiltLimitInputChange() {
  if (cloudTilt_Limit_input != ProgramTiltThreshold) {
    ProgramTiltThreshold = cloudTilt_Limit_input;
    Serial.println("TiltLimitChanged");
  }
}
```

```
//18
float bME_Index;
```

```
float cloudLiveVibrations;

float cloudTilt_Limit_input;

float cloudTiltX;

float cloudTiltY;

float cloud_Vibration_Upper_Limit_input;

CloudTemperatureSensor bME_Temp;

int bME_Humidity;

int cloudHumidity_Upper_Limit_input;

int cloudTemp_Lower_Limit_input;

int cloudTemp_Upper_Limit_input;

bool cloudAQNormal;

bool cloudAQVeryBad;

bool cloudAQVeryGood;

bool cloudHumidityWarning;

bool cloudTempWarning;

bool cloudTiltWarning;

bool cloudTooLowTempWarning;

bool cloudVibrationWarning;
```

4.1 Описание на програмата

Проектът е написан на C++ и е записано върху микроконтролера, като използва платформата Arduino IoT Cloud за изграждането на проекта, свързаност и управление. Arduino IoT Cloud осигурява вграждането на облачни променливи (дефинирани в thingProperties.h), които позволяват обмен на информация от кода в таблото за наблюдение, в реално време. Основната функционалност на програмата включва прочитането на данни от сензори, обработка на тези данни, при

необходимост, активиране на предупредителни сигнали и визуализация на резултатите на таблото.

4.2 Функционалности на програмата

Програмата е структурирано в блокове. Така подобрява четливостта и леснотата на изграждането му. Това са всичките предназначения на следните блокове:

1. В началото на програмата се включват всички библиотеки. За сензорите , тези на Adafruit са най-пасващи към целите на програмата да работи и прегледност.

Adafruit_MPU6050.h [13]

Предоставя функции за комуникация с MPU6050 сензора чрез I2C интерфейс. Позволява конфигурация на акселерометъра и жироскопа, както и четене на данни за ускорение, ъглово завъртане и температура.

Adafruit_BME680.h [14]

Осигурява API за взаимодействие с BME688 сензора чрез I2C интерфейс. Позволява четене на температура, влажност, налягане и газово съпротивление, както и настройка на параметри като честота на измерване и газов нагревател.

thingProperties.h

Съдържа автоматично генерирани функции и настройки, които свързват устройството с Arduino IoT Cloud. Обработва обмена на данни между локалните променливи и облачната платформа.

Wire.h

Библиотека за работа с I2C протокол. Позволява изпращане и получаване на данни между микроконтролера и други устройства.

2. Деклариране на обекти за BME680 и MPU6050 сензорите.

3. Дефиниране на константи за приблизителното изчисление на качеството на въздуха.

4. Програмата създава променливи с начални стойности, но който се подават информацията от облачните променливи във функциите нормирани от 15 до 19. Те

[13] [10]„Adafruit MPU6050 Sensor Library,“ [Онлайн]. Available: https://adafruit.github.io/Adafruit_MPU6050/html/class_adafruit___m_p_u6050.html.
[14] [11] „Adafruit BME680 Library,“ [Онлайн]. Available: https://adafruit.github.io/Adafruit_BME680/html/class_adafruit___b_m_e680.html.

включват прагове за вибрации, наклон, температура и влажност, както и за индикаторите за качеството на въздуха.

5. В setup() се извършва първоначалната инициализация на програмата. Стартира се серийната комуникация за дебъг информация, проверява се дали сензорите BME688 и MPU6050 са налични и се конфигурират техните параметри. Освен това се настройва връзката с Arduino IoT Cloud, за да се осигури синхронизация между локалните променливи и облачните дефинирани в thingProperties.h.

6. В loop() се изпълняват основните задачи на програмата. На всяка итерация се обновяват данните в Arduino IoT Cloud, като се изпращат текущите стойности и предупреждения. След това се извикват функциите за засичане на вибрации и наклони, четене на данни от BME680 сензора и обработка на предупрежденията. Използват се кратки закъснения, за да се осигури стабилност и синхронизация между отделните операции.

7. Функцията detectVibrationAndTilt() обработва данни от сензора MPU6050, за да засече вибрации и наклони. Изчислява общото ускорение и го сравнява с предварително зададен праг за вибрации. Ако се засече вибрация, се записва на облачното предупреждение. Също се изчисляват наклоните по осите X и Y, като се проверява дали те надвишават зададения праг. Ако се установи продължителен наклон, се активира предупреждение и се отпечатва подробна информация.

8. Функцията readBmeSensor() чете текущите данни от сензора BME688, като извършва измервания на температура, влажност и газово съпротивление. Ако четенето е успешно, тези стойности се съхраняват в съответните променливи и се изчислява приблизително качеството на въздуха. След това функцията извиква помощни методи, които проверяват и задават предупреждения за въздух, температура и влажност въз основа на събраните данни. Ако четенето не е успешно, се отпечатва съобщение за грешка в серийния монитор.

9-11.Функциите raiseAQIndicators(), raiseTemperatureIndicators() и raiseHumidityIndicators() са отговорни за анализ на данните от BME688 и задаване на съответните предупреждения на таблото. raiseAQIndicators() проверява индекса на качеството на въздуха и светва за "Много добро", "Нормално" или "Зле" качество. raiseTemperatureIndicators() следи за температурни отклонения спрямо зададен диапазон, активира предупреждения за висока или ниска температура.

raiseHumidityIndicators() проверява дали влажността надвишава зададения праг, ако трябва активира предупреждение. Резултатите от всички проверки се изпращат към облака.

12. Функцията SerialMonitorDebuggingBME() показва измерванията от BME680 (температура, влажност, налягане, газ) в серийния монитор.

13. Функцията invertCloudWarnings() обръща всички записани болей стойности на индикаторите. Това е за да свети таблото в червено вместо зелено на таблото.

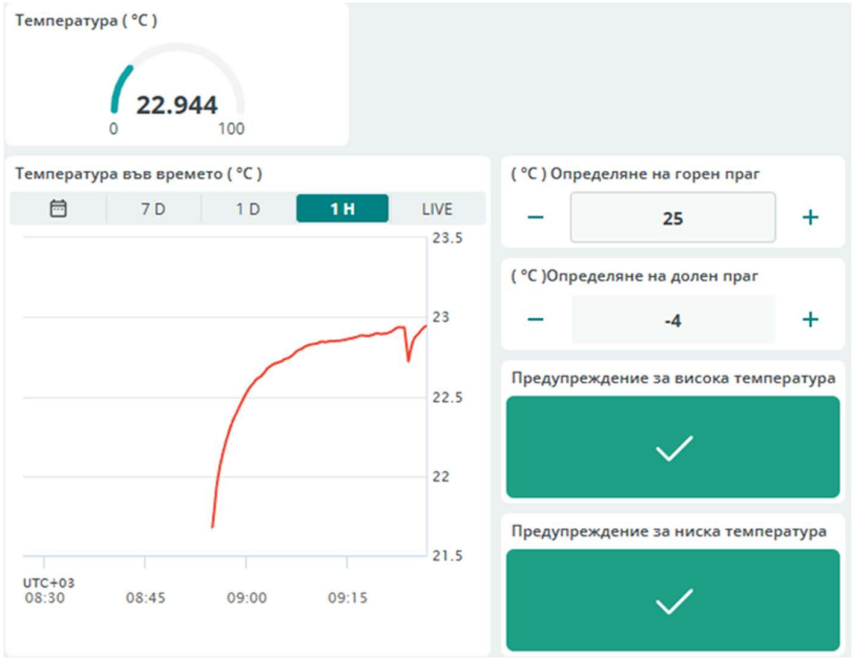
14. Функцията float calculateAQI(float gasResistance, float humidity) изчисляване на качеството е приблизителен.

15-19. Функциите позволяват промяна на програмните настройки (температурни граници, вибрационни прагове, влажност и наклон) в реално време чрез облака. Всяка функция проверява дали новите стойности от облака са различни от текущите и ако е така, обновява съответния параметър.

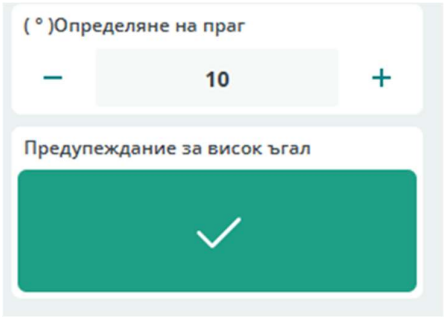
18- Дефинираните облачни променливи в thingProperties.h файл.

ГЛАВА ВТОРА. ПРАКТИЧЕСКА ЧАСТ

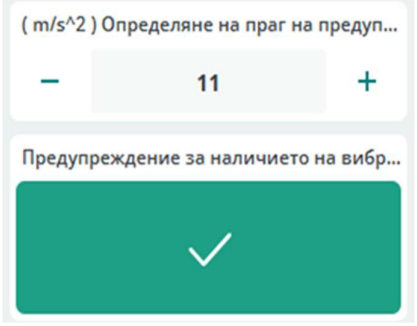
1. Облачен интерфейс – екран на наблюдател



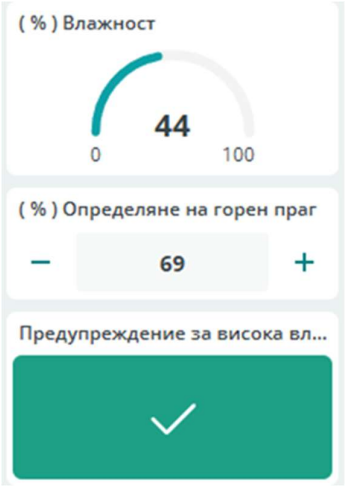
Фиг. 10 Температурни визуализиращи елементи, бутони и индикатори на таблото



Фиг. 11 Стъпков бутон и индикатор за състоянието на ъгъла на таблото



Фиг. 12 Стъпков бутон и индикатор за липсата вибрации

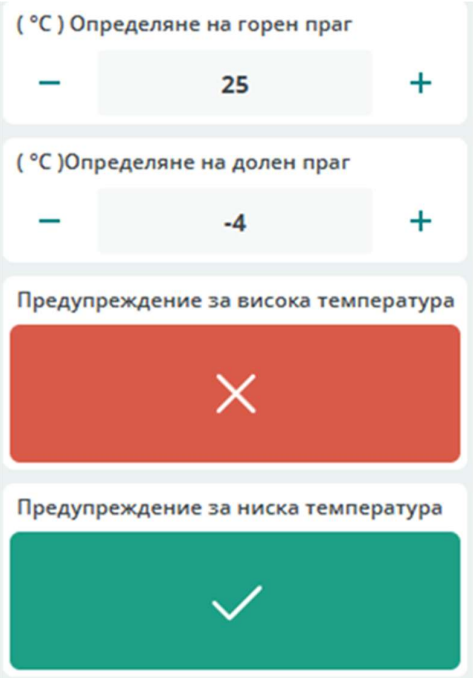


Фиг. 13 Стъпков бутон и индикатор влага



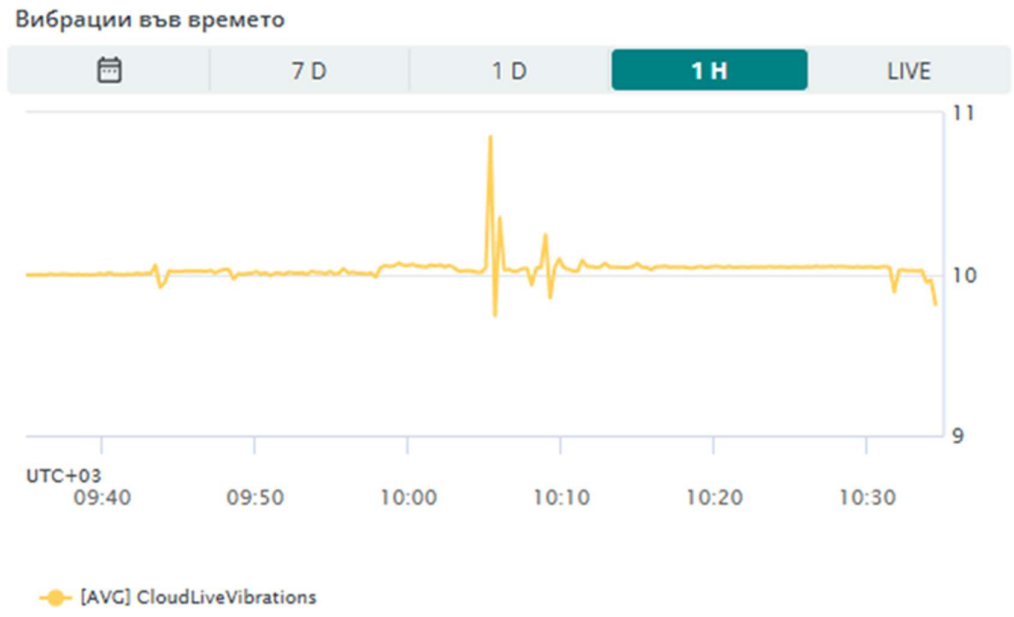
Фиг. 14 Индикатори за състояние на въздуха

От Фиг.10 до Фиг.14 са представени всички основни индикатори. За всеки параметър може да му се определи праговете (с изключение качеството на въздуха). Ако премине в не благоприятни условия ще промени индикатора да свети в червено за лошо. (пример в фиг 15.) Създадени са на платформата на Arduino, която получава стойностите от проекта и ги изобразява.

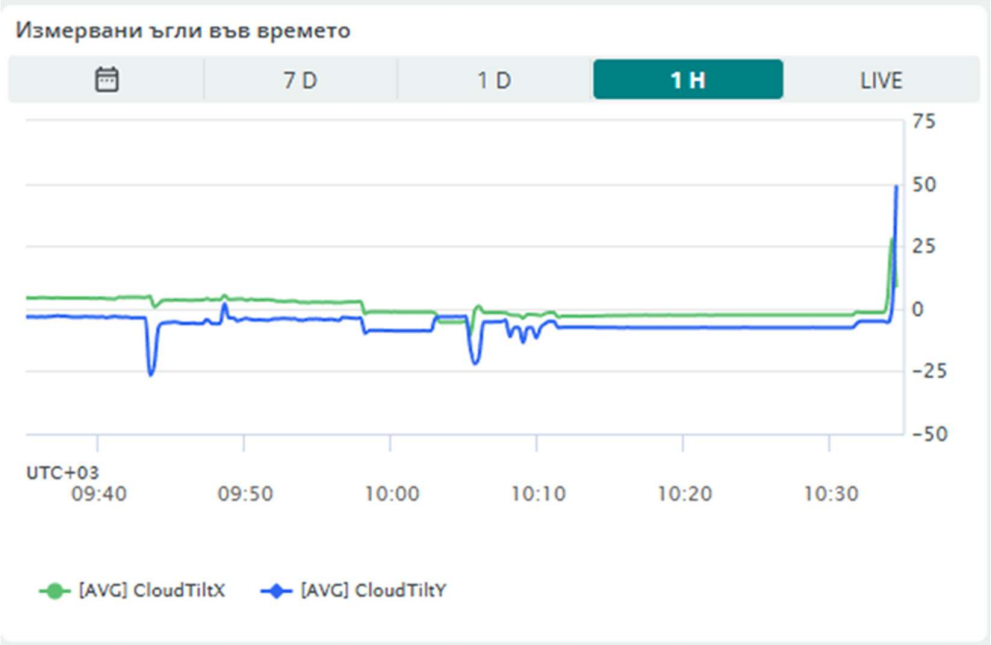


Фиг. 15

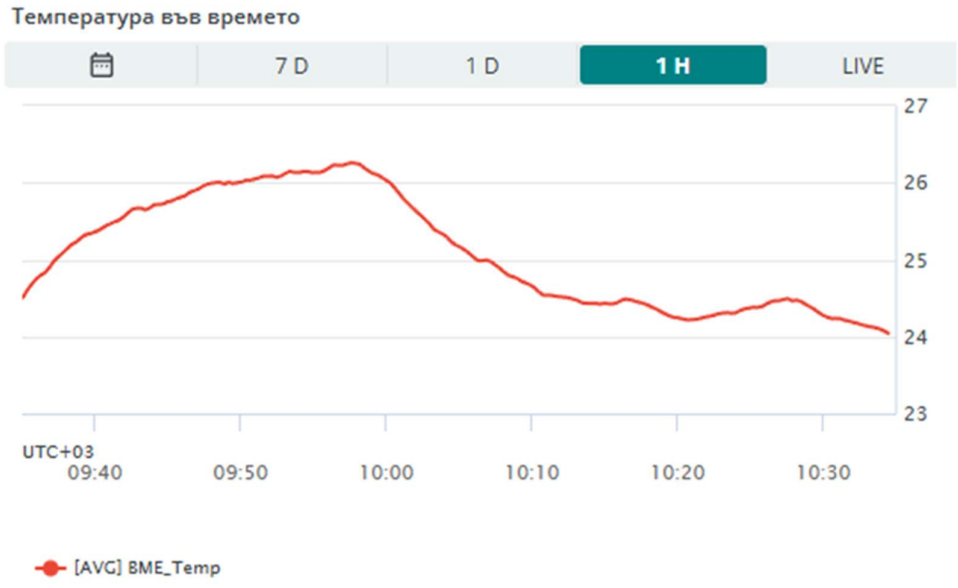
1.2 Облачен интерфейс – екран с данни във времето



Фиг. 16 Графично изобразяване на вибрации във времето.



Фиг. 17 Графично изобразяване на измервани ъгли във времето



Фиг. 18 Графично изобразяване на температура във времето



Фиг. 19 Графично изобразяване на влага във времето

Фиг. 16 до Фиг.19 се вижда графично изобразени стойностите на параметри в продължението на един час назад във времето.

2. Инструкции за работа

1. Изтеглете от Google Play Store апликацията Arduino IoT Remote

2. Инсталирайте я.

3. Влезте в апликацията с Login Name: DPUser и Password: Service1!

4. Заредете таблото на наблюдавана индустриална среда: “Simeon_Bogoev_12D_22

5. На екрана ще виждате, графично изобразени, наблюдаваните параметри от наблюдаваната индустриална среда. В таблото „Simeon_Bogoev_12D_22“ индикаторите със зелен флаг показват, че параметрите са в нормите зададени, като нормални. Червеният флаг, съответно, показва че, има задействана аларма и че, един или повече параметри са излезли извън от програмираните, като нормални граници.

6. Другото табло „Graphs” има графично изобразяване на параметрите от таблото „Simeon_Bogoev_12D_22“

Нужно е локална мрежа да в обхвата на проекта с име „ДипломенПроект“ и парола „admin123”.

Заключение

В резултат на реализирания проект бе разработена работеща IoT система за дистанционен мониторинг на индустриално оборудване. Интегрирането на температурни, влагомерни, газоанализни, вибрационни и ъглови сензори, свързани към микроконтролер по I2C шина, доказа своята надеждност и точност при събиране на данни. Комуникацията по Wi-Fi към облачната платформа протича стабилно, а уеб интерфейсът осигурява интуитивен и бърз достъп до текущите и исторически измервания. Тестването в работни условия потвърди качеството на предаваните данни.

Разработената платформа улеснява навременното откриване на отклонения в работата на машините, позволява прогнозна поддръжка и минимизира риска от аварии и непланирани спирания. Централизираният достъп до информация дава възможност на операторите да вземат по-информирани решения и да оптимизират графика на техническите проверки. Това води до по-ефективно използване на ресурси и намалени оперативни разходи.

Източници

[1] Arduino, „Arduino UNO R4 WiFi Datasheet (ABX00087),“ TDK, 2023.

[2] R. Electronics, „RA4M1 Datasheet Rev.1.10,“ 2023.

[3] E. Systems, „ESP32-S3-MINI Datasheet v1.35,“ 2020.

[4] B. Sensortec, „BME688 Sensor Datasheet Rev.1.3,“ 2024.

[5] I. (TDK), „MPU-6050 Register Map Rev.4.2,“ 2013.

[6] L. M. Engineers, „MPU-6050 Arduino Tutorial,“ LME Tutorials, [Онлайн]. Available: https://lastminuteengineers.com/mpu-6050-accelerometer-and-gyroscope-arduino-tutorial/.

[7] M. Electronics. [Онлайн]. Available: https://ar.mouser.com/new/renesas/renesas-ra8m1-32-bit-mcus/.

[8] Espressif. [Онлайн]. Available: https://www.espressif.com/en/news/UNO_R4_WiFi_ESP32-S3.

[9] J. Wu, „A Basic Guide to I2C,“ Texas Instruments, 2022..

[10] „Adafruit MPU6050 Sensor Library,“ [Онлайн]. Available: https://adafruit.github.io/Adafruit_MPU6050/html/class_adafruit___m_p_u6050.html.

[11] „Adafruit BME680 Library,“ [Онлайн]. Available: https://adafruit.github.io/Adafruit_BME680/html/class_adafruit___b_m_e680.html.